

综合实战项目效果评估

(1) 模型微调评估总结报告

本部分总结综合实战项目微调结果，数据来自评估程序 4.evaluate_model.py 的输出。

模型评估结果整理报告

一、总体对比结果

- 测试样本总数：200
- 基座模型胜出：13 条 (6.5%)
- 微调模型胜出：182 条 (91.0%)
- 平局：5 条 (2.5%)

整体来看，微调模型在绝大多数样本中优于基座模型，胜率达到 **91.0%**。

二、核心指标对比

指标	基座模型	微调模型	提升
BERTScore F1	0.5395	0.6467	+0.1072
字数合规率	0.1350	0.9950	+0.8600
拒绝准确率 (红区)	0.8889	1.0000	+0.1111
RAG 忠实度	0.9855	0.7609	-0.2246
综合得分	0.4756	0.7501	+0.2746

关键观察

1. 语义质量明显提升

- BERTScore 提升 **0.1072**
- 说明微调模型生成内容更接近参考答案

2. 格式控制能力显著增强

- 字数合规率从 **13.5% → 99.5%**

- 说明微调数据成功强化了输出规范

3. 安全拒绝能力提升

- 红区拒绝准确率达到 100%
- 表明模型能够稳定拒绝违规请求

4. RAG 忠实度下降

- 从 0.9855 → 0.7609
- 说明微调后模型更倾向于生成而不是严格引用检索结果

这是当前系统**唯一明显下降**的指标。

三、分场景表现

类别	样本数	基座模型	微调模型	微调胜率
 普通	122	0.3695	0.7275	100%
 RAG	69	0.6600	0.7819	73.9%
 红区	9	0.4996	0.8129	100%

各类别分析

1 普通对话

- 提升最明显
- 微调模型 **全部**胜出

说明微调数据对 **客服知识学习**非常有效

2 RAG 问答

- 微调仍然更好（胜率 73.9%）
- 但 **RAG 忠实度下降**

原因通常是：

- 微调数据强化了生成能力
 - 模型更容易“发挥”而不是引用检索内容
-

3 红区安全

- 拒绝能力明显增强
- 微调模型 100% 正确拒绝

说明 安全策略数据有效

四、总体结论

1 微调模型整体性能显著提升

- 胜率 91%
- 综合得分 +0.2746

说明微调已经成功学习客服任务。

2 三个能力显著提升

任务能力

- 更贴近标准答案
- 更符合客服语气

格式控制

- 输出长度严格受控

安全策略

- 红区问题可稳定拒绝
-

3 唯一需要改进的方向

RAG 忠实度下降

可能原因：

1 微调数据中

RAG 样本比例不足

2 微调目标强调

回答质量 > 检索引用

3 模型学会

“合理编造”

五、优化建议（非常关键）

如果这是一个 **RAG 客服系统**，建议：

1 增加 RAG 训练数据

建议比例：

普通数据：RAG 数据：红区数据

5 : 4 : 1

2 强化引用约束

训练数据示例：

【系统检索结果】

订单号：123456

状态：已发货

用户：订单在哪里？

助手：

根据系统检索结果，订单 123456 已发货。

3 加入“禁止编造”样本

例如：

如果检索结果没有相关信息，请回答：

"根据当前检索结果无法确认该信息"

六、最终评价

当前模型已经达到：

高质量客服模型

优势：

- 回答质量高
- 输出规范
- 安全性强

唯一不足：

- **RAG 引用严格性**

但这是 可通过数据增强快速解决的问题。

(2) 模型量化评估总结报告 (FP16 vs AWQ)

本部分总结综合实战项目量化结果, 数据来自评估脚本 7.evaluate_quantization.py 的输出。

一、回答质量对比 (200 条样本)

结果	数量	比例
----	----	----

FP16 更优	48	24.0%
---------	----	-------

AWQ 更优	64	32.0%
--------	----	-------

平局	88	44.0%
----	----	-------

整体来看, 两种模型**质量基本相当**, AWQ 略有优势。

二、质量指标对比

指标	FP16 均值	AWQ 均值	变化
BERTScore F1	0.6255	0.6330	+0.0076
字数合规率	0.9900	0.9950	+0.0050
拒绝准确率 (红区)	1.0000	1.0000	0
RAG 忠实度	0.7850	0.7850	0
综合质量得分	0.7384	0.7439	+0.0056

结论

- AWQ 与 FP16 **质量基本一致**
- AWQ 略微提高:
 - 语义相似度 (BERTScore)
 - 输出格式合规性
- RAG 忠实度和安全拒绝能力完全一致**

三、分类任务表现

类别	样本数	FP16	AWQ	AWQ 胜率
 普通问答	122	0.7124	0.7168	35.2%
 RAG 问答	69	0.7773	0.7855	29.0%
 红区安全	9	0.7918	0.7922	11.1%

结论

- 普通问答与 RAG 任务中 AWQ 略优
- 安全拒绝能力 完全一致

四、推理性能对比

模型资源占用

指标	FP16	AWQ	变化
模型体积	3.4 GB	1.7 GB	↓ 48.9%
显存占用	3504 MB	1790 MB	↓ 48.9%

结论

AWQ 将：

- 模型体积减少约 49%
- 显存占用减少约 49%

显著提升部署效率。

推理性能

指标	FP16	AWQ	变化
首 Token 延迟	42.6 ms	62.9 ms	×1.48
生成速度	23.7 tok/s	15.9 tok/s	×0.67

结论

当前测试环境中：

- AWQ 生成速度低于 FP16
- 首 Token 延迟更高

主要原因：

- fuse_layers=False
 - awq_ext 未安装
 - 算子融合未启用
-

五、综合结论

1 质量

AWQ 与 FP16 质量等价

- 综合质量差异： +0.0056
- 在统计误差范围内

因此：

AWQ 量化不会明显降低模型质量

2 资源效率

AWQ 显著降低资源消耗：

- 模型体积 ↓ 48.9%
- 显存占用 ↓ 48.9%

适合：

- 单卡部署
 - 边缘设备
 - GPU 资源紧张环境
-

3 当前性能

当前 AWQ 推理速度略慢：

- 生成速度: **0.67× FP16**
- 首 Token 延迟: **1.48×**

但这是运行环境问题。

4 生产环境预期

在 vLLM + AWQ 部署后:

通常会得到:

- 推理速度 $\geq$ FP16
- 显存占用 $\downarrow$ 50%
- 吞吐量显著提升

六、关键注意事项

1 首 Token 延迟已进行预热, 消除了 CUDA 冷启动影响。

2 当前 AWQ 运行配置:

`fuse_layers = False`

原因:

`awq_ext` 未安装

导致:

- 算子融合未生效
- 推理速度下降

3 生产环境建议:

使用

`vLLM + AWQ`

或

`AutoAWQ + fuse_layers=True`

即可获得正常性能。

 质量评估使用：

BERTScore (本地 FP16)

无需联网。

七、最终判断

AWQ 量化结论

维度	结果
----	----

质量	≈ FP16
----	--------

显存	↓ 49%
----	-------

模型大小	↓ 49%
------	-------

当前速度	略慢
------	----

生产速度	预计更快
------	------

因此：

AWQ 是推荐的生产部署方案。